VALORANT ESPORTS TOURNAMENT ECOSYSTEM

DBMS Project

IILM UNIVERSITY

B.Tech Computer Science & Engineering

Academic Year 2025 - 26

Group Members

Abhay Kumar · 2410030695 · **Group Leader**

Dushyant Kumar · 2410030677 · Member

Vaishnavi Sharma · 2410030681 · Member

Deepak Kumar · 2410030660 · Member

1. Problem Statement

Grassroots and collegiate Valorant tournaments are typically managed through spreadsheets. This approach works until it doesn't — the moment a player registers under two teams, a K/D ratio is calculated incorrectly, or bracket results from different organizers contradict each other, the entire system collapses. There is no structure enforcing data integrity, no single source of truth for match data, and no clean mechanism to generate accurate leaderboards across a full tournament.

This project designs and implements a relational database for a Valorant tournament management system. The schema covers tournaments, teams, players, matches (with per-map results for BO3/BO5 formats), and individual player statistics per map. The SQL queries address real operational questions: standings, top performers, agent pick rates, and map win rates — all backed by a normalized, constraint-enforced schema.

2. Database Schema

The schema consists of six interrelated tables. Tournaments is the root entity — every match belongs to one. A Match record holds the two competing teams and the overall result. Individual map results live in Match_Maps so BO3 and BO5 formats are supported properly. Player_Stats ties each player's performance to a specific map within a specific match.

Table 1: Tournaments

Column	Type	Constraint	Notes
tournament_id	INT	PRIMARY KEY, AUTO_INCREMENT	
name	VARCHAR(100)	NOT NULL	e.g. VCT Challengers 2025
region	VARCHAR(50)	NOT NULL	APAC, EMEA, Americas
format	VARCHAR(20)	NOT NULL	Single / Double Elimination
start_date	DATE	NOT NULL	
end_date	DATE	NULL	Null until concluded
prize_pool	DECIMAL(12,2)		In USD

Table 2: Teams

Column	Type	Constraint	Notes
team_id	INT	PRIMARY KEY, AUTO_INCREMENT	
team_name	VARCHAR(100)	NOT NULL, UNIQUE	
region	VARCHAR(50)	NOT NULL	
founded_date	DATE		

Table 3: Players

Column	Type	Constraint	Notes
player_id	INT	PRIMARY KEY, AUTO_INCREMENT	
ign	VARCHAR(50)	NOT NULL, UNIQUE	In-game name
real_name	VARCHAR(100)		
primary_role	VARCHAR(30)	NOT NULL	Duelist / Controller / Sentinel / Initiator
country	VARCHAR(50)		
team_id	INT	FK -> Teams(team_id)	Current team

Table 4: Matches

Column	Type	Constraint	Notes
match_id	INT	PRIMARY KEY, AUTO_INCREMENT	
tournament_id	INT	FK -> Tournaments	
team_a_id	INT	FK -> Teams(team_id)	
team_b_id	INT	FK -> Teams(team_id)	
match_format	VARCHAR(5)	NOT NULL	BO1 / BO3 / BO5
match_date	DATETIME	NOT NULL	
winner_id	INT	FK -> Teams(team_id)	NULL if match is ongoing

Table 5: Match_Maps

Column	Type	Constraint	Notes
map_result_id	INT	PRIMARY KEY, AUTO_INCREMENT	
match_id	INT	FK -> Matches(match_id)	
map_number	INT	NOT NULL	1, 2, 3 …
map_name	VARCHAR(50)	NOT NULL	Ascent, Bind, Haven, etc.
team_a_score	INT	NOT NULL	Rounds won
team_b_score	INT	NOT NULL	Rounds won
map_winner_id	INT	FK -> Teams(team_id)	

Table 6: Player_Stats

Column	Type	Constraint	Notes
stat_id	INT	PRIMARY KEY, AUTO_INCREMENT	
match_id	INT	FK -> Matches(match_id)	
map_result_id	INT	FK -> Match_Maps	Per map, per match
player_id	INT	FK -> Players(player_id)	
agent_played	VARCHAR(50)	NOT NULL	Jett, Omen, Killjoy, etc.
kills	INT	DEFAULT 0	
deaths	INT	DEFAULT 0	
assists	INT	DEFAULT 0	
acs	DECIMAL(6,2)	DEFAULT 0	Average Combat Score
headshot_pct	DECIMAL(5,2)	DEFAULT 0	Range: 0.00 – 100.00

3. SQL Queries

All 28 queries are grouped by type. Each entry includes a problem statement, the SQL, and the expected output. Queries target MySQL 8.0+. Window functions (OVER()) require MySQL 8.0 or higher. DELETE sequences follow FK dependency order — child rows are removed before parent rows.

A. DML — INSERT

Query 1 — INSERT

Problem: *Add a new tournament to the database.*

```
INSERT INTO Tournaments (name, region, format, start_date, prize_pool)
VALUES ('VCT Challengers 2025', 'APAC', 'Double Elimination', '2025-06-01',
50000.00);
```

Output:

```
Query OK, 1 row affected
tournament_id = 1  (AUTO_INCREMENT)
```

Query 2 — INSERT

Problem: *Register a new team.*

```
INSERT INTO Teams (team_name, region, founded_date)
VALUES ('Team Phantom', 'APAC', '2023-01-15');
```

Output:

```
Query OK, 1 row affected
team_id = 1  (AUTO_INCREMENT)
```

Query 3 — INSERT

Problem: *Add a player and assign them to Team Phantom.*

```
INSERT INTO Players (ign, real_name, primary_role, country, team_id)
VALUES ('ShadowX', 'Arjun Mehta', 'Duelist', 'India', 1);
```

Output:

```
Query OK, 1 row affected
player_id = 1  (AUTO_INCREMENT)
```

Query 4 — INSERT

Problem: *Record a BO3 match result between two teams.*

```
INSERT INTO Matches (tournament_id, team_a_id, team_b_id, match_format,
match_date, winner_id)
VALUES (1, 1, 2, 'BO3', '2025-06-05 14:00:00', 1);
```

Output:

```
Query OK, 1 row affected
match_id = 1  (AUTO_INCREMENT)
```

Query 5 — INSERT

Problem: *Log a player's stats for a specific map in a match.*

```
INSERT INTO Player_Stats
  (match_id, map_result_id, player_id, agent_played, kills, deaths, assists, acs,
headshot_pct)
VALUES (1, 1, 1, 'Jett', 24, 10, 5, 312.50, 38.20);
```

Output:

```
Query OK, 1 row affected
```

B. DML — UPDATE

Query 6 — UPDATE

Problem: *Correct a player's headshot percentage after a data entry error.*

```
UPDATE Player_Stats
SET headshot_pct = 42.50
WHERE stat_id = 1;
```

Output:

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1
```

Query 7 — UPDATE

Problem: *Transfer a player to a different team.*

```
UPDATE Players
SET team_id = 3
WHERE player_id = 1;
```

Output:

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1
```

Query 8 — UPDATE

Problem: *Set the end date of a tournament once it finishes.*

```
UPDATE Tournaments
SET end_date = '2025-06-20'
WHERE tournament_id = 1;
```

Output:

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1
```

C. DML — DELETE

Query 9 — DELETE

Problem: *Remove a player who has left the competitive scene.*

Player_Stats rows are deleted first to avoid a foreign key violation.

```
-- Step 1: Remove all stat records for this player
DELETE FROM Player_Stats WHERE player_id = 5;

-- Step 2: Remove the player record
DELETE FROM Players WHERE player_id = 5;
```

Output:

```
Query OK, 8 rows affected   (Player_Stats)
Query OK, 1 row affected    (Players)
```

Query 10 — DELETE

Problem: *Delete all data for a cancelled tournament, respecting FK dependency order.*

```
-- Step 1: Remove player stats for all matches in this tournament
DELETE FROM Player_Stats
  WHERE match_id IN (SELECT match_id FROM Matches WHERE tournament_id = 2);

-- Step 2: Remove map results
DELETE FROM Match_Maps
  WHERE match_id IN (SELECT match_id FROM Matches WHERE tournament_id = 2);

-- Step 3: Remove matches
DELETE FROM Matches WHERE tournament_id = 2;

-- Step 4: Remove the tournament
DELETE FROM Tournaments WHERE tournament_id = 2;
```

Output:

```
Query OK   (Player_Stats cleaned)
Query OK   (Match_Maps cleaned)
Query OK   (Matches cleaned)
Query OK, 1 row affected  (Tournaments)
```

D. SELECT WITH CONDITIONS

Query 11 — SELECT (BETWEEN)

Problem: *Get all matches played within a specific date range.*

```
SELECT match_id, match_format, match_date, winner_id
FROM   Matches
WHERE  match_date BETWEEN '2025-06-01' AND '2025-06-10'
ORDER  BY match_date ASC;
```

Output:

```
match_id | match_format | match_date          | winner_id
---------|--------------|---------------------|----------
       1 | BO3          | 2025-06-05 14:00:00 |         1
       2 | BO3          | 2025-06-07 10:00:00 |         2
       3 | BO5          | 2025-06-09 16:00:00 |         1
```

Query 12 — SELECT (WHERE + AND)

Problem: *Find all Duelist players from India.*

```
SELECT player_id, ign, real_name, country
FROM   Players
WHERE  primary_role = 'Duelist'
  AND  country = 'India';
```

Output:

```
player_id | ign     | real_name   | country
----------|---------|-------------|--------
        1 | ShadowX | Arjun Mehta | India
        4 | Blaze7  | Rohan Singh | India
```

Query 13 — SELECT (LIKE)

Problem: *Search for any player whose IGN contains 'Ace'.*

```
SELECT player_id, ign, primary_role, team_id
FROM   Players
WHERE  ign LIKE '%Ace%';
```

Output:

```
player_id | ign       | primary_role | team_id
----------|-----------|--------------|--------
        7 | AceStorm  | Controller   |       2
       12 | AceHuntr  | Initiator    |       4
```

E. JOIN QUERIES

Query 14 — INNER JOIN (2 tables)

Problem: *List every player with their current team name and region.*

```
SELECT p.ign, p.primary_role, t.team_name, t.region
FROM   Players p
JOIN   Teams t ON p.team_id = t.team_id
ORDER  BY t.team_name, p.ign;
```

Output:

```
ign       | primary_role | team_name    | region
----------|--------------|--------------|-------
AceStorm  | Controller   | Team Eclipse | EMEA
Nexus     | Sentinel     | Team Eclipse | EMEA
Blaze7    | Duelist      | Team Phantom | APAC
ShadowX   | Duelist      | Team Phantom | APAC
```

Query 15 — INNER JOIN (3 tables)

Problem: *Show each match with tournament name, both team names, and the winner.*

```
SELECT m.match_id,
       tn.name          AS tournament,
       ta.team_name     AS team_a,
       tb.team_name     AS team_b,
       m.match_format,
       tw.team_name     AS winner
FROM   Matches m
JOIN   Tournaments tn ON m.tournament_id = tn.tournament_id
JOIN   Teams ta        ON m.team_a_id    = ta.team_id
JOIN   Teams tb        ON m.team_b_id    = tb.team_id
LEFT JOIN Teams tw     ON m.winner_id    = tw.team_id;
```

Output:

```
match_id | tournament           | team_a       | team_b       | format | winner
---------|----------------------|--------------|--------------|--------|----------
       1 | VCT Challengers 2025 | Team Phantom | Team Eclipse | BO3    | Team
Phantom
       2 | VCT Challengers 2025 | Team Eclipse | Team Nova    | BO3    | Team
Eclipse
```

Query 16 — LEFT JOIN

Problem: *List all teams and their total match wins, including teams with zero wins.*

```
SELECT   t.team_name, COUNT(m.winner_id) AS wins
FROM     Teams t
LEFT JOIN Matches m ON m.winner_id = t.team_id
GROUP BY t.team_id, t.team_name
ORDER BY wins DESC;
```

Output:

```
team_name     | wins
--------------|-----
Team Phantom  |    4
Team Eclipse  |    2
Team Nova     |    1
Team Cipher   |    0
```

F. AGGREGATE FUNCTIONS

Query 17 — COUNT / AVG / MAX

Problem: *Get each player's average ACS, highest kill game, and total maps played.*

```
SELECT   p.ign,
         COUNT(ps.stat_id)       AS maps_played,
         ROUND(AVG(ps.acs), 2)   AS avg_acs,
         MAX(ps.kills)           AS highest_kills
FROM     Players p
JOIN     Player_Stats ps ON ps.player_id = p.player_id
GROUP BY p.player_id, p.ign
ORDER BY avg_acs DESC
LIMIT 5;
```

Output:

```
ign       | maps_played | avg_acs | highest_kills
----------|-------------|---------|---------------
ShadowX   |          12 |  298.40 |            28
AceStorm  |          10 |  281.75 |            25
Blaze7    |           9 |  265.20 |            22
Nexus     |          11 |  243.60 |            19
```

Query 18 — GROUP BY + HAVING

Problem: *Find players who averaged above 250 ACS across at least 5 maps.*

```
SELECT   p.ign, p.primary_role,
         ROUND(AVG(ps.acs), 2)  AS avg_acs,
         COUNT(ps.stat_id)      AS maps_played
FROM     Players p
JOIN     Player_Stats ps ON ps.player_id = p.player_id
GROUP BY p.player_id, p.ign, p.primary_role
HAVING   AVG(ps.acs) > 250
  AND    COUNT(ps.stat_id) >= 5
ORDER BY avg_acs DESC;
```

Output:

```
ign       | primary_role | avg_acs | maps_played
----------|--------------|---------|------------
ShadowX   | Duelist      |  298.40 |          12
AceStorm  | Controller   |  281.75 |          10
Blaze7    | Duelist      |  265.20 |           9
```

Query 19 — SUM + GROUP BY

Problem: *Calculate total kills, deaths, and overall K/D ratio per team in a tournament.*

```
SELECT   t.team_name,
         SUM(ps.kills)                                    AS total_kills,
         SUM(ps.deaths)                                   AS total_deaths,
         ROUND(SUM(ps.kills) / NULLIF(SUM(ps.deaths),0), 2) AS kd_ratio
FROM     Teams t
JOIN     Players p      ON p.team_id    = t.team_id
JOIN     Player_Stats ps ON ps.player_id = p.player_id
JOIN     Matches m      ON m.match_id   = ps.match_id
WHERE    m.tournament_id = 1
GROUP BY t.team_id, t.team_name
ORDER BY kd_ratio DESC;
```

Output:

```
team_name     | total_kills | total_deaths | kd_ratio
--------------|-------------|--------------|----------
Team Phantom  |         342 |          198 |     1.73
Team Eclipse  |         290 |          225 |     1.29
Team Nova     |         210 |          278 |     0.76
```

G. SUBQUERIES

Query 20 — Subquery (WHERE IN)

Problem: *Get all players who played in matches won by Team Phantom.*

```
SELECT DISTINCT p.ign, p.primary_role
FROM   Players p
JOIN   Player_Stats ps ON ps.player_id = p.player_id
WHERE  ps.match_id IN (
         SELECT match_id FROM Matches
         WHERE  winner_id = (
                  SELECT team_id FROM Teams WHERE team_name = 'Team Phantom'
                )
       );
```

Output:

```
ign      | primary_role
---------|-------------
ShadowX  | Duelist
Blaze7   | Duelist
Vortex   | Initiator
ShieldUp | Sentinel
```

Query 21 — Subquery (Scalar)

Problem: *Find the most-picked agent across the tournament.*

```
SELECT   agent_played, COUNT(*) AS times_picked
FROM     Player_Stats
WHERE    match_id IN (
           SELECT match_id FROM Matches WHERE tournament_id = 1
         )
GROUP BY agent_played
ORDER BY times_picked DESC
LIMIT 1;
```

Output:

```
agent_played | times_picked
-------------|-------------
Jett         |           38
```

Query 22 — Subquery (Correlated)

Problem: *List players whose ACS on Ascent is above the Ascent average.*

```
SELECT p.ign, ps.acs, mm.map_name
FROM   Player_Stats ps
JOIN   Players    p  ON p.player_id       = ps.player_id
JOIN   Match_Maps mm ON mm.map_result_id  = ps.map_result_id
WHERE  mm.map_name = 'Ascent'
  AND  ps.acs > (
         SELECT AVG(ps2.acs)
         FROM   Player_Stats ps2
         JOIN   Match_Maps mm2 ON mm2.map_result_id = ps2.map_result_id
         WHERE  mm2.map_name = 'Ascent'
       )
ORDER BY ps.acs DESC;
```

Output:

```
ign       |    acs | map_name
----------|--------|----------
ShadowX   | 312.50 | Ascent
AceStorm  | 295.00 | Ascent
Blaze7    | 280.20 | Ascent
```

Query 23 — Subquery (EXISTS)

Problem: *List teams that have won at least one match in the tournament.*

```
SELECT t.team_name, t.region
FROM   Teams t
WHERE  EXISTS (
         SELECT 1 FROM Matches m
         WHERE  m.winner_id     = t.team_id
           AND  m.tournament_id = 1
       );
```

Output:

```
team_name     | region
--------------|-------
Team Phantom  | APAC
Team Eclipse  | EMEA
```

H. ADVANCED QUERIES

Query 24 — Agent Pick Rate (Window Function)

Problem: *Show each agent's pick count and percentage share across the tournament.*

```
SELECT agent_played,
       COUNT(*) AS total_picks,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pick_rate_pct
FROM   Player_Stats
GROUP BY agent_played
ORDER BY total_picks DESC;
```

Output:

```
agent_played | total_picks | pick_rate_pct
-------------|-------------|---------------
Jett         |          38 |         18.63
Omen         |          32 |         15.69
Killjoy      |          27 |         13.24
Sova         |          24 |         11.76
Skye         |          20 |          9.80
```

Query 25 — Map Win Rate per Team

Problem: *Calculate each team's win rate on every map to identify their strongest maps.*

```
SELECT   t.team_name,
         mm.map_name,
         COUNT(*) AS played,
         SUM(CASE WHEN mm.map_winner_id = t.team_id THEN 1 ELSE 0 END) AS won,
         ROUND(
           SUM(CASE WHEN mm.map_winner_id = t.team_id THEN 1 ELSE 0 END)
           * 100.0 / COUNT(*), 1
         ) AS win_pct
FROM     Teams t
JOIN     Matches m  ON (m.team_a_id = t.team_id OR m.team_b_id = t.team_id)
JOIN     Match_Maps mm ON mm.match_id = m.match_id
GROUP BY t.team_id, t.team_name, mm.map_name
ORDER BY t.team_name, win_pct DESC;
```

Output:

```
team_name     | map_name | played | won | win_pct
--------------|----------|--------|-----|--------
Team Phantom  | Ascent   |      6 |   5 |    83.3
Team Phantom  | Bind     |      5 |   3 |    60.0
```

```
Team Phantom  | Haven    |      4 |   2 |    50.0
Team Eclipse  | Icebox   |      5 |   4 |    80.0
```

Query 26 — Tournament Standings

Problem: *Full standings: wins, losses, maps won, and net round difference.*

```
SELECT
  t.team_name,
  SUM(CASE WHEN m.winner_id = t.team_id THEN 1 ELSE 0 END)  AS wins,
  SUM(CASE WHEN m.winner_id != t.team_id THEN 1 ELSE 0 END) AS losses,
  SUM(CASE WHEN mm.map_winner_id = t.team_id THEN 1 ELSE 0 END) AS maps_won,
  SUM(CASE WHEN m.team_a_id = t.team_id
       THEN mm.team_a_score - mm.team_b_score
       ELSE mm.team_b_score - mm.team_a_score END) AS round_diff
FROM   Teams t
JOIN   Matches m   ON (m.team_a_id = t.team_id OR m.team_b_id = t.team_id)
                   AND m.tournament_id = 1
JOIN   Match_Maps mm ON mm.match_id = m.match_id
GROUP BY t.team_id, t.team_name
ORDER BY wins DESC, round_diff DESC;
```

Output:

```
team_name     | wins | losses | maps_won | round_diff
--------------|------|--------|----------|------------
Team Phantom  |    4 |      1 |        9 |        +42
Team Eclipse  |    3 |      2 |        7 |        +18
Team Nova     |    2 |      3 |        5 |         -8
Team Cipher   |    1 |      4 |        3 |        -52
```

I. SQL VIEW — PLAYER LEADERBOARD

A View stores the leaderboard query so it can be called like a table. Front-end dashboards can query this view directly without re-running the full aggregation each time.

Query 27 — CREATE VIEW

Problem: *Create a reusable player leaderboard view with ACS, KDA, and headshot statistics.*

```
CREATE VIEW vw_player_leaderboard AS
SELECT
  p.player_id, p.ign, p.primary_role, t.team_name,
  COUNT(ps.stat_id)                                AS maps_played,
  ROUND(AVG(ps.acs), 2)                            AS avg_acs,
  SUM(ps.kills)                                    AS total_kills,
  SUM(ps.deaths)                                   AS total_deaths,
  SUM(ps.assists)                                  AS total_assists,
  ROUND(SUM(ps.kills) / NULLIF(SUM(ps.deaths),0), 2) AS kd_ratio,
  ROUND(AVG(ps.headshot_pct), 2)                   AS avg_hs_pct
FROM   Players p
JOIN   Teams t        ON t.team_id    = p.team_id
JOIN   Player_Stats ps ON ps.player_id = p.player_id
GROUP BY p.player_id, p.ign, p.primary_role, t.team_name;
```

Output:

```
View 'vw_player_leaderboard' created successfully.
```

Query 28 — SELECT from View

Problem: *Pull the top 5 players by average ACS from the leaderboard view.*

```
SELECT ign, team_name, avg_acs, kd_ratio, avg_hs_pct
FROM   vw_player_leaderboard
ORDER BY avg_acs DESC
LIMIT 5;
```

Output:

```
ign       | team_name    | avg_acs | kd_ratio | avg_hs_pct
----------|--------------|---------|----------|------------
ShadowX   | Team Phantom |  298.40 |     2.10 |      38.20
AceStorm  | Team Eclipse |  281.75 |     1.88 |      34.50
Blaze7    | Team Phantom |  265.20 |     1.72 |      31.80
Nexus     | Team Eclipse |  243.60 |     1.45 |      28.90
Vortex    | Team Phantom |  238.10 |     1.38 |      26.40
```